



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Estructura y Programación de Computadoras (1503)

Profesora: Ariel Adara Mercado Martínez
Semestre 2026-2

Bitácora IA

Grupo: 05

Alumnos:

Arcos Garduño Itzcoatl Ajax
García Aguilar José Iván
Martínez Chavez Alexis
Martínez Méndez Cedrik Alexis
Motta Reyes Emmanuel Alberto

Índice

1. Herramientas IA utilizadas	2
2. Diseño previo del equipo	2
3. Registro de sesiones	3
4. Resumen de código generado vs manual	8
5. Resultados de pruebas	9
6. Observaciones finales	9

1. Herramientas IA utilizadas

Herramienta	Versión	Uso principal
Claude (Anthropic)	Sonnet 4.6	Revisión de diseño, generación de código, depuración

2. Diseño previo del equipo

Antes de utilizar herramientas de IA, el equipo elaboro un pseudocódigo completo del sistema dividido en 9 módulos:

- Estructura general del sistema (main)
- Lexer y tokenización
- Parser y representación interna
- Tabla de símbolos
- Ensamblador de una pasada con fixups
- Ensamblador de dos pasadas
- Encoder IA-32 con ModRM y SIB
- Formato objeto propio
- Mini Linker

Este pseudocódigo fue elaborado por el equipo como punto de partida. Posteriormente se utilizo IA para revisar la arquitectura, identificar posibles problemas de diseño y sugerir mejoras antes de proceder a la implementación.

Refinamientos identificados por la IA sobre el diseño del equipo:

Separar claramente las responsabilidades entre encoder y assembler. Agregar el campo 'defined' a la tabla de símbolos para distinguir símbolos pendientes de los ya resueltos. Incluir soporte explícito para relocalaciones R32 y PC32 desde el diseño.

3. Registro de sesiones

Sesion 1 — Revision de arquitectura y planificacion

Fecha: 02/06/2026

Contexto: El equipo presento su pseudocódigo inicial a la IA para revisión y retroalimentación antes de comenzar la implementación.

Prompts utilizados

- Revisa este pseudocódigo del sistema ensamblador-linker e identifica posibles problemas de diseño o módulos incompletos
- El equipo propone esta estructura de archivos, que ajustes recomiendas para cumplir con los requisitos del proyecto
- Explica como debe fluir la información entre los módulos desde el archivo ASM hasta el binario final

Lo que genero la IA

- Retroalimentación sobre el diseño propuesto por el equipo
- Sugerencias de refinamiento para la tabla de símbolos y el encoder
- Confirmación del flujo de datos entre módulos

Modificaciones manuales

- El equipo incorporo las sugerencias al diseño final antes de codificar

Errores encontrados:

Ninguno en esta sesión.

Sesion 2 — Implementacion de la arquitectura base

Fecha: 02/06/2026

Contexto: Con el diseño validado, el equipo utilizo IA para acelerar la implementacion de los módulos base siguiendo la arquitectura acordada.

Prompts utilizados

- Basándose en el diseño del equipo, implementa en C el lexer para IA-32 con soporte para registros de 32 y 8 bits, instrucciones, directivas y manejo de comentarios
- Implementa el parser siguiendo el diseño del equipo con soporte para los 7 modos de direccionamiento IA-32 incluyendo SIB completo
- Implementa la tabla de símbolos con soporte para símbolos globales, externos y detección de redefiniciones
- Implementa el ensamblador de una y dos pasadas con manejo de fixups y relaciones

- Implementa el encoder IA-32 con construcción de ModRM y SIB, registros de 32 y 8 bits, y soporte para MOVZX
- Diseña e implementa el formato objeto acordado por el equipo con encabezado, tabla de secciones, símbolos y relaciones
- Implementa el linker con fusión de secciones, resolución de símbolos externos y aplicación de relaciones
- Implementa el punto de entrada principal con soporte para modos ensamblar, linkear, ensamblar-linkear y salida hexadecimal --hex

Lo que genero la IA

- Implementación completa de los 8 módulos del sistema en C
- Archivos de cabecera con definición de estructuras y prototipos
- Makefile con reglas de compilación, limpieza y pruebas
- Soporte para registros de 8 bits: AL, AH, BL, BH, CL, CH, DL, DH
- Instrucción MOVZX para extensión de byte a 32 bits sin signo
- Opción --hex para generación de archivo hexadecimal legible

Modificaciones manuales

- Creación y organización de archivos en el repositorio de GitHub
- El equipo revisó cada módulo contra el diseño original

Errores encontrados

Error 1: Ubicación incorrecta de archivo

Causa: src/lexer.c fue creado accidentalmente dentro de include/src/ en lugar de src/

Sintoma: El archivo aparecía en la ruta incorrecta
include/src/lexer.c

Solución: Corrección manual de la ruta desde la interfaz de GitHub y eliminación del archivo duplicado

Sesión 3 — Casos de prueba y herramientas auxiliares

Fecha: 03/06/2026

Contexto: El equipo definió los casos de prueba necesarios y utilizó IA para implementarlos junto con las herramientas de validación.

Prompts utilizados

- El equipo definió estas 8 categorías de prueba, implementa los archivos ASM: MOV inmediato, saltos, referencias adelantadas, EXTERN, CALL, relocalaciones, múltiples módulos y SIB
- Implementa en Python un script de hexdump para inspección de binarios
- Implementa un script de comparación byte a byte contra NASM para validación de correctitud del encoder
- Implementa un script de pruebas automatizadas con reporte de resultados

Lo que generó la IA

- 8 casos de prueba cubriendo todos los requisitos del proyecto
- 3 scripts Python para inspección, comparación y automatización

Modificaciones manuales

- El equipo revisó y ajustó los casos de prueba para cubrir sus escenarios específicos

Errores encontrados:

Ninguno en esta sesión.

Sesión 4 — Depuración e integración

Fecha: 03/06/2026

Contexto: Al ejecutar las pruebas, el equipo identificó dos bugs en el parser y los reportó a la IA para diagnóstico y corrección.

Prompts utilizados

- El parser entra en loop infinito al procesar SECTION .text, el equipo identificó que el problema está en parse_operand, confirma y corrige

- MOV [resultado_suma], EAX falla porque parse_memory no acepta identificadores simbólicos, proporciona la corrección

Lo que genero la IA

- Confirmación del diagnostico del equipo
- Correccion completa de src/parser.c con los dos fixes integrados

Modificaciones manuales

- Aplicación de correcciones via terminal en Codespaces
- Verificación manual de correctitud mediante pruebas
- Commit y push de cambios

Errores encontrados

Error 1: Loop infinito en parser con SECTION .text

Causa: .TEXT tokenizado como TOKEN_IDENTIFIER sin caso en parse_operand, el parser no avanzaba

Sintoma: Miles de líneas: Error línea 6: operando invalido (encontrado: '.TEXT')

Solución: Agregar caso TOKEN_IDENTIFIER en parse_operand con advance() forzado

Error 2: Error en direccionamiento con identificadores simbolicos

Causa: parse_memory solo aceptaba registros o números dentro de corchetes

Sintoma: Error línea 22: se esperaba ']' (encontrado: 'RESULTADO_SUMA')

Solución: Agregar caso TOKEN_IDENTIFIER en parse_memory

Error 3: Script run_tests.py se colgaba indefinidamente

Causa: subprocess.run sin timeout cuando el ensamblador entraba en loop infinito

Sintoma: El script mostraba 'Pruebas de dos pasadas' y no avanzaba

Solución: Detener con Ctrl+C y corregir el bug del parser primero

Error 4: NASM no disponible en Codespaces

Causa: Repositorios de paquetes no actualizados en el entorno

Sintoma: `E: Unable to locate package nasm`

Solución: Ejecutar `apt-get update` primero y luego instalar NASM

Sesión 5 — Validación final y documentación

Fecha: 06/06/2026

Contexto: El equipo realizo la validación final del proyecto comparando la salida del ensamblador contra NASM y preparo la documentación de entrega.

Prompts utilizados

- Genera el reporte técnico completo del proyecto en formato Word
- Genera una guía de explicación del proyecto que cubra objetivo, funcionamiento modulo a modulo y resultados esperados
- Actualiza la bitácora con todas las sesiones del proyecto

Lo que genero la IA

- Reporte tecnico en formato .docx con todas las secciones requeridas
- Guia de explicación con analogías y ejemplos visuales
- Bitácora completa del proyecto en formato .docx

Modificaciones manuales

- El equipo reviso y ajusto el contenido de los documentos
- Actualización del apartado de organización del equipo en el reporte

Errores encontrados

Error 1: Conflicto de Git al hacer push

Causa: El repositorio remoto tenia commits mas recientes que el Codespace local

Sintoma: `error: failed to push some refs – Updates were rejected`

Solución: `git rebase --abort` seguido de `git push --force` para conservar los cambios locales

4. Resumen de código generado vs manual

Modulo	Diseñado por el equipo	Generado por IA	Ajustado manualmente
include/lexer.h	100%	Implementacion	0%
src/lexer.c	100%	Implementacion	5%
include/parser.h	100%	Implementacion	0%
src/parser.c	100%	Implementacion	15% (bugs)
include/symbols.h	100%	Implementacion	0%
src/symbols.c	100%	Implementacion	0%
include/assembler.h	100%	Implementacion	0%
src/assembler.c	100%	Implementacion	0%
include/encoder.h	100%	Implementacion	10%
src/encoder.c	100%	Implementacion	10%
include/object.h	100%	Implementacion	0%
src/object.c	100%	Implementacion	0%
include/linker.h	100%	Implementacion	0%
src/linker.c	100%	Implementacion	0%
src/main.c	100%	Implementacion	5%
Makefile	100%	Implementacion	0%
Scripts Python	80%	Implementacion	20%
Casos de prueba	100%	Implementacion	5%

5. Resultados de pruebas

Prueba	Resultado
test_mov.asm — todos los modos de direccionamiento	PASS
test_jumps.asm — saltos y referencias adelantadas	PASS
test_call.asm — CALL y manejo de pila	PASS
test_extern.asm — símbolos externos	PASS
test_sib.asm — byte SIB con todas las escalas	PASS
test_multimodule — enlazado de dos módulos	PASS
test_reg8.asm — registros de 8 bits y MOVZX	PASS

Comparacion con NASM (instrucciones simples)	Bytes idénticos
Compilación sin warnings con -Wall -Wextra	0 warnings

6. Observaciones finales

- El proceso mas efectivo fue que el equipo diseñara primero y la IA implementara después, en lugar de delegar el diseño completamente
- Los prompts mas efectivos especificaban el contexto técnico completo y referencia en el diseño previo del equipo
- Los bugs encontrados fueron identificados por el equipo durante las pruebas y confirmados por la IA en el diagnostico
- La validación byte a byte contra NASM fue propuesta por el equipo como criterio de correctitud objetivo
- La arquitectura modular elegida por el equipo facilito la depuración al poder aislar y probar cada componente de forma independiente
- Los conflictos de Git fueron una oportunidad de aprendizaje sobre el manejo de ramas en equipos